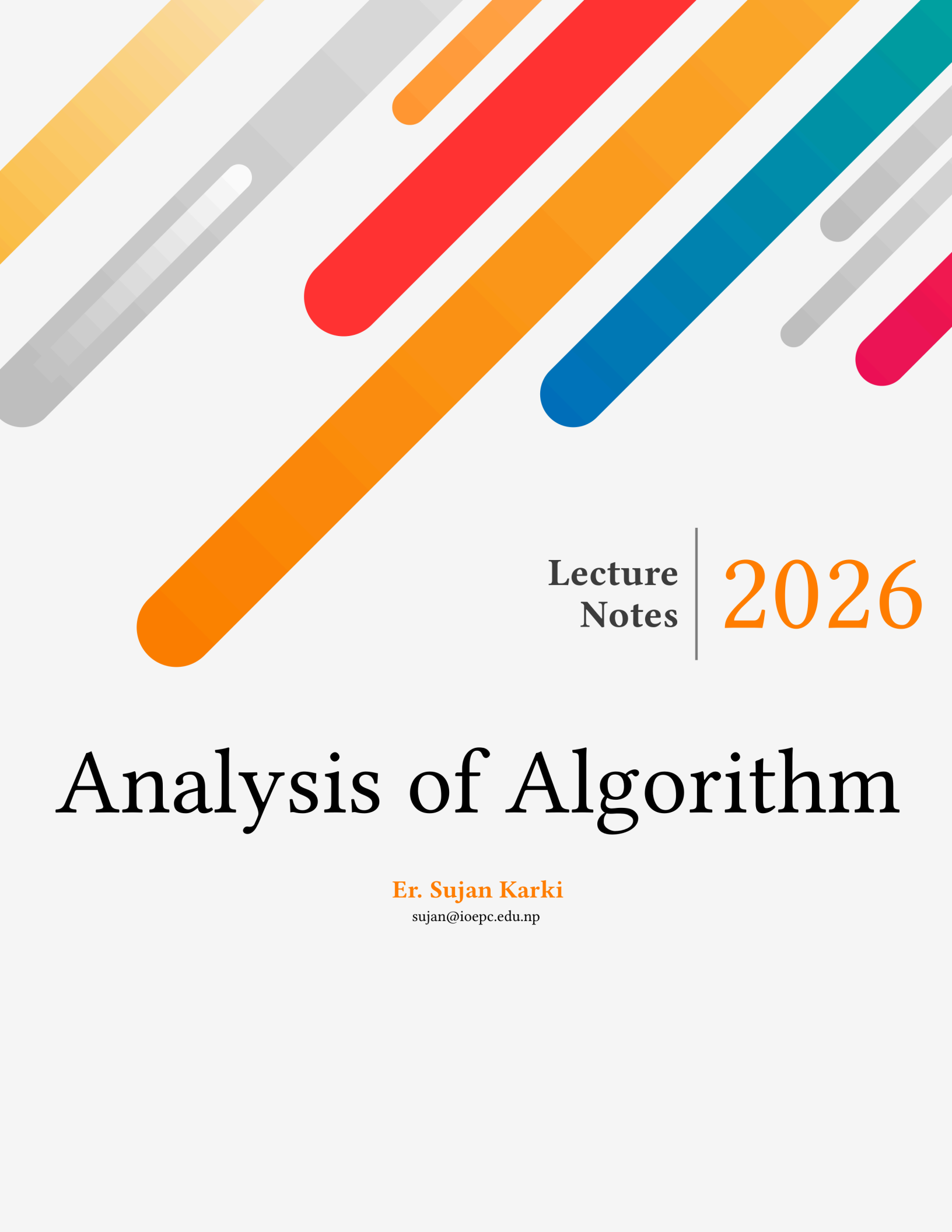




Lecture
Notes

2026

Analysis of Algorithm

Er. Sujan Karki

sujan@ioepc.edu.np

Chapter 1

Introduction to Algorithm Analysis

1.1 Algorithm and Its Properties

1.1.1 What is an Algorithm?

Definition 1.1.1

An **algorithm** is a well-defined computational procedure that takes some value, or a set of values, as *input* and produces some value, or a set of values, as *output*. It is a finite sequence of unambiguous instructions that solves a given problem.

In other words, an algorithm is a step-by-step recipe that transforms inputs into outputs through a finite number of clearly defined steps.

1.1.2 Properties of an Algorithm

A procedure qualifies as a good algorithm only if it satisfies the following properties:

1. Correctness

The algorithm must produce the correct and expected output for all valid inputs.

2. Finiteness

The algorithm must terminate after a finite number of steps.

3. Absence of Ambiguity (Unambiguity)

Each step must be clear and precise, having exactly one meaning.

4. Sequence (Orderliness)

The steps must be arranged in a specific, logical order.

5. Well-defined Input and Output

The algorithm must clearly specify what it accepts and what it returns.

6. Feasibility (Practicability)

Each instruction must be executable using the available resources.

7. Flexibility (Modifiability)

A good algorithm should be easy to modify or update if the problem changes.

8. Efficiency

Measured in terms of *time complexity* (how fast it runs) and *space complexity* (how much memory it uses).

9. Language Independence

It should be describable without using the syntax of any specific programming language.

1.2 The RAM Model

1.2.1 What is the RAM Model?

Definition 1.2.1

The **RAM (Random Access Machine) Model** is a theoretical model used to design and analyze algorithms. It acts as an abstract computer that simplifies real hardware so that algorithm analysis becomes easier and more consistent. It helps us measure *Time Complexity* and *Space Complexity*.

1.2.2 Why Do We Need the RAM Model?

Real computers have many complexities: caches, pipelines, parallel CPUs, operating-system interruptions, memory hierarchy, and other hardware-specific behaviors. These make measuring the exact running time of an algorithm difficult. To avoid these details we require a *machine-independent* way – the RAM Model – a simplified ideal computer with predictable behavior.

1.2.3 Assumptions of the RAM Model

1. **Constant Time for Simple Operations:** Every basic operation (addition, subtraction, multiplication, division, comparison, assignment) takes exactly 1 time step. This makes it easy to count the number of steps in an algorithm.
2. **Loops and Subroutines Are Not Simple Operations:** They are collections of instructions. The total time depends on the number of iterations or calls, not just one unit.
3. **Memory Access Takes One Time Step:** Reading from or writing to a memory location takes exactly 1 time step, independent of the location of the memory cell (random access).
4. **Unlimited Memory:** The model assumes no shortage of memory, so algorithms can theoretically use as much memory as needed.

1.2.4 Structure of the RAM Model

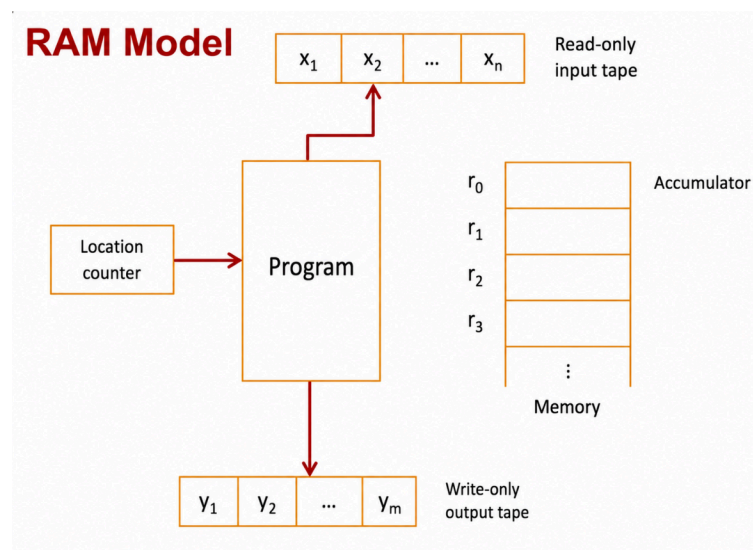


Figure 1.1: RAM Model

The RAM model is a generic machine with five components: a read-only *input tape*, a write-only *output tape*, a *program*, a *program/location counter*, and *memory* (registers).

When analyzing, we count the **primitive operations**: assigning a value to a variable, performing an arithmetic operation, indexing into an array, comparing two numbers, calling a method, and returning from a method.

1.3 Computational Complexity

Definition 1.3.1

Computational complexity refers to the study of the resources required to solve a computational problem. The two main resources are *time* and *space*.

1.3.1 Time Complexity

Time complexity quantifies the amount of time taken by an algorithm to run as a function of the length of the input.

More input $\Rightarrow$ More time

If you have n ingredients and you must mix each one once, the time grows proportionally with n .

1.3.2 Space Complexity

Space complexity quantifies the amount of memory taken by an algorithm to run as a function of the length of the input.

More input $\Rightarrow$ More space

If you have an array of n numbers to store, you need memory for all n .

Although time and space depend on hardware, OS, and processors, we ignore these factors during analysis and express complexity using **asymptotic notations**: $O(n)$, $\Theta(n)$, and $\Omega(n)$.

1.4 Detailed Analysis of Algorithms

The process of finding the complexity of a given algorithm by counting the number of steps and the number of memory references, using the RAM model, is called **detailed analysis**. For detailed time complexity of algorithm we count the number of steps and finally add all steps and calculate their big oh notation. Similarly for detailed analysis of space complexity we count the number of memory references and finally add all steps and find their big oh notation.

Example 1: Constant Time

```
1  int a;           // Variable declaration generally ignored in asymptotic analysis.
2  a = 10;          // 1 time step
3  a++;             // 1 time step
4  int b = a-20;    // 1 + 1 time step
```

Total time steps: $1 + 1 + 2 = 4$

Time Complexity: $O(1)$ (constant time)

Memory: $a, b \Rightarrow 2 \text{ units} \Rightarrow O(1) \text{ space}$

Example 2: Linear Time

```
1  for (i=1; i<=n; i++) { // 1, (n+1), n
2      printf("%d", i);   // n
3  }
```

Total time steps: $1 + (n + 1) + n + n = 2 + 3n$

Time Complexity: $O(n)$ (linear time)

Memory: $i \Rightarrow 1 \text{ unit} \Rightarrow O(1) \text{ space}$

Example 3: Linear Time (While Loop)

```

1  int i = 0;           // 1 time step
2  while (i < n) {      // (n+1) time steps
3      printf("%d", i); // n
4      i = i+1;         // n
5  }
```

Total time steps: $1 + (n + 1) + n + n = 2 + 3n$

Time Complexity: $O(n)$ **Space:** $O(1)$

Example 4: Linear Time (for Loop with Step Size)

```

1  for (i=1; i<=n; i=i+2) { // 1, (n/2)+1, n/2
2      printf("%d", i);    // n/2
3  }
```

Total time steps: $1 + (n/2 + 1) + (n/2) + n/2$

Time Complexity: $O(n)$ **Space:** $O(1)$

Example 5: Linear Time (Function Call)

```

1  int sum(int n) {
2      int total = 0;           // 1
3      for (int i=1; i<=n; i++) // 1, (n+1), n
4          total += i*i;        // n
5      return total;           // 1
6  }
```

Total time steps: $1 + 1 + (n + 1) + n + n + 1$

Time Complexity: $O(n)$ **Space:** $O(1)$ (total, i)

Example 6: Quadratic Time

```

1  for (i=1; i<=n; i++) { // 1, (n+1), n
2      for (j=1; j<=n; j++) { // n, n(n+1), n*n
3          printf("%d", i*j); // n*n
4      }
5  }
```

Total time steps: $1 + (n + 1) + n + n + n * (n + 1) + n * n + n * n = 2 + 3n + 3n^2$

Time Complexity: $O(n^2)$ (quadratic time)

Memory: $i, j \Rightarrow 2 \text{ units} \Rightarrow O(1) \text{ space}$

Example 7: Triangular Nested Loops

```

1  for (int i=1; i<=n; i++) // 1, (n+1), n
2      for (int j=1; j<=i; j++) // n, (1+1)+(2+1)+(3+1)...+(n+1) = (n(n+1)/2) + n, n(n+1)/2
3          printf("%d %d\n", i, j);
```

loop runs $1 + (n+1) + n + n + \left(\frac{n(n+1)}{2} + n\right) + \frac{n(n+1)}{2} + \frac{n(n+1)}{2}$

Time Complexity: $O(n^2)$

Space: $O(1)$

Example 8: Quadratic Growth of a Counter

```
1 int p = 0;
2 for (i=1; p<=n; i++)
3     p = p + i;
```

i	p
1	0 + 1
2	1 + 2
3	1 + 2 + 3
4	1 + 2 + 3 + 4
	
k (let)	1 + 2 + 3 + 4 + ... + k

Here $p = 1 + 2 + 3 + \dots + k = \frac{k(k+1)}{2}$. Stopping condition: $p > n$, so $\frac{k(k+1)}{2} > n \Rightarrow k \approx \sqrt{n}$.

Time Complexity: $O(\sqrt{n})$

Space: $O(1)$

Example 9: Multiplicative Logarithmic Time

```
1 for (int i=1; i<n; i=i*2)
2     Statement;
```

i
1
$1 * 2 = 2$
$2 * 2 = 2^2$
....
2^k

Values of i : $1, 2, 2^2, \dots, 2^k$. Loop stops when $2^k \geq n$, so $k = \log_2 n$.

Time Complexity: $O(\log_2 n)$

Space: $O(1)$

Example 10: Divisive Logarithmic Time

```
1 for (int i=n; i>1; i=i/2)
2     Statement;
```

i
n
$n/2$
$n/2^2$
..
$n/2^k$

Values of i : $n, n/2, n/4, \dots, n/2^k$. Loop stops when $i \leq 1$,

Now, $n/2^k \leq 1 \Rightarrow n \leq 2^k \Rightarrow$ So, $k = \log_2 n$.

Time Complexity: $O(\log_2 n)$

Space: $O(1)$

Example 11: Logarithmic Time with Multiplication

```

1      for (int i=0; i<n; i++)           // 1, n+1, n
2      for (int j=1; j<n; j=j*2)       // n, (n(log n)+1), n log n
3          Statement;                  // n * log n

```

Values of j: $1, 2, 4, \dots, 2^k$. The loop stops when $2^k \geq n$, so $k = \log_2 n$.

Total time steps: $1 + (n+1) + n + n + (n(\log n) + 1) + (n \log n) + (n \log n)$

Time Complexity: $O(n \log n)$

Space: $O(1)$

Example 12: Factorial Calculation

```

1  int i, fact = 1, n;
2  printf("Enter a number: ");
3  scanf("%d", &n);
4  for (i=1; i<=n; i++)
5      fact = fact * i;
6  printf("Factorial = %d", fact);
7  return 0;

```

Total time steps: $1 + 1 + 1 + 1 + (n+1) + n + 2n + 1 + 1$

Time Complexity: $O(n)$

Space: $O(1)$ (i, fact, n)

Example 13: Fibonacci Sequence

```

1  int fibonacci(int n) {
2      int a=0, b=1, next;
3      for (int i=2; i<=n; i++) {
4          next = a + b;
5          a = b;
6          b = next;
7      }
8      return b;
9  }

```

Total time steps: $1 + 1 + n + (n-1) + 2(n-1) + (n-1) + (n-1) + 1$

Time Complexity: $O(n)$ **Space:** $O(1)$ (n, a, b, next)

Example 14: Summation of an Array

```

1  int Sum(int a[], int n) {
2      int s = 0;           // 1
3      for (int i=1; i<=n; i++) // 1, n+1, n
4          s = s + a[i];     // 2n
5      return s;            // 1
6  }

```

Total time steps: $1 + 1 + (n+1) + n + 2n + 1$

Time Complexity: $O(n)$

Space Complexity: i, s are used = $1 + 1 \Rightarrow O(1)$ space

Total Memory: $O(n)$ (for input array) + $O(1)$ (for all other variables) = $O(n)$

Example 15: Space Complexity with Order of Input

```
1  int arr[n];           // array of size n
2  for (int i=0; i<n; i++)
3      arr[i] = i;
```

The array `arr[n]` requires memory proportional to the input size n . **Space Complexity:** $O(n)$

Example 16: Space Complexity with 2D Array

```
1  int n = 5;
2      int matrix[n][n];
3  for (int i = 0; i < n; i++) {
4      for (int j = 0; j < n; j++) {
5          matrix[i][j] = i * n + j;
6      }
7  }
```

Time Complexity: $O(n^2)$ (due to nested loops)

The 2D array `matrix[n][n]` requires memory proportional to n^2 .

Space Complexity: $O(n^2)$

1.5 Amortized Complexity

Amortized complexity is a method of analyzing the *average time per operation* over a sequence of operations, even if some individual operations are expensive. The idea is to spread the cost of expensive operations over many operations so that the average cost per operation remains low.

There are three techniques used in amortized analysis:

- Aggregate analysis
- Accounting method
- Potential method

1.5.1 Concept of Aggregate Analysis

Aggregate analysis is the simplest method of amortized analysis. It applies a **sum-and-divide** approach:

- Determine the worst-case cost of the entire sequence of operations, $T(n)$.
- Divide this cost by the number of operations n .

$$\text{Amortized operation cost} = \frac{T(n)}{n}$$

This gives one uniform cost for all operations.

Its limitations:

- (1) it works only when $T(n)$ is easy to compute, and
- (2) all operations receive the same amortized cost — no differentiation between cheap and expensive operations.

We analyze the operation sequence:

`push(1), push(2), push(3), pop(), multipop(2), push(4), pop()`

In the aggregate method, we compute the total cost of a sequence of n operations and divide by n to obtain the amortized cost per operation.

Operation	Actual Cost	Notes
push(1)	1	push 1 element
push(2)	1	push 1 element
push(3)	1	push 1 element
pop()	1	pop 1 element
multipop(2)	2	pop 2 elements
push(4)	1	push 1 element
pop()	1	pop 1 element

$$\text{Total cost} = 1 + 1 + 1 + 1 + 2 + 1 + 1 = 8$$

$$\text{Amortized cost per operation} = \frac{\text{Total cost}}{\text{Number of operations}} = \frac{8}{7} \approx 1.14 \approx O(1)$$

Even though some operations like `multipop` seem expensive individually, aggregate analysis shows that the average cost per operation is $O(1)$.

1.5.2 Accounting Method

The accounting method of amortized analysis gets its name because it borrows ideas from financial accounting. Instead of using the exact actual cost of each operation, we assign an artificial cost, called the **amortized cost**, to every operation.

- Some operations are assigned **more** than their actual cost. The extra amount is stored as **credit**.
- Some operations are assigned **less** than their actual cost. The previously saved credit is used to pay the remaining cost.

Key Points

- Each operation is assigned an amortized cost.
- If actual cost < amortized cost → extra goes to credit.
- If actual cost > amortized cost → use credit to pay the difference.
- Helps smooth out expensive operations over a sequence of operations.

Example: Stack with `multipop(k)`

Consider a stack supporting:

- `push(x)` → actual cost = 1
- `pop()` → actual cost = 1
- `multipop(k)` → actual cost = $\min(k, \text{current stack size})$

Assigned amortized costs:

- `push(x)` → 2
- `pop()` → 0 (paid by credit)
- `multipop(k)` → 0 (paid by credit)

Operation	Actual Cost	Amortized Cost	Credit After
push(1)	1	2	1
push(2)	1	2	2
push(3)	1	2	3
pop()	1	0	2
multipop(2)	2	0	0
push(4)	1	2	1
pop()	1	0	0

$$\text{Total Actual Cost} = 1 + 1 + 1 + 1 + 2 + 1 + 1 = 8$$

$$\text{Total Amortized Cost} = 2 + 2 + 2 + 0 + 0 + 2 + 0 = 8$$

- Total amortized cost $\geq$ total actual cost
- Amortized cost per operation $= \frac{8}{7} \approx O(1)$

1.5.3 Potential Method

The Potential Method assigns amortized costs to operations based on a **potential function**, which measures the “stored work” or “future cost” in a data structure.

- Each state of the data structure has a potential value $\Phi(D)$.
- Amortized cost of an operation:

$$\text{Amortized Cost} = \text{Actual Cost} + \Delta\Phi, \quad \Delta\Phi = \Phi(\text{after}) - \Phi(\text{before})$$

If the operation increases potential $\rightarrow$ it “stores credit” for future operations. If the operation decreases potential $\rightarrow$ it “uses credit” stored previously.

Potential Method with Stack

Let $\Phi(\text{stack}) = \text{number of elements in the stack}$. Each element represents potential energy to pay for pops later. Initially, the stack is empty, so $\Phi_0 = 0$.

$$\text{Amortized Cost} = \text{Actual Cost} + (\Phi_{\text{after}} - \Phi_{\text{before}})$$

Operation	Stack Before	Actual	Φ Before	Stack After	Φ After	$\Delta\Phi$	Amort.
push(1)	[]	1	0	[1]	1	+1	2
push(2)	[1]	1	1	[1, 2]	2	+1	2
push(3)	[1, 2]	1	2	[1, 2, 3]	3	+1	2
pop()	[1, 2, 3]	1	3	[1, 2]	2	-1	0
multipop(2)	[1, 2]	2	2	[]	0	-2	0
push(4)	[]	1	0	[4]	1	+1	2
pop()	[4]	1	1	[]	0	-1	0

- Total actual cost $= 1 + 1 + 1 + 1 + 2 + 1 + 1 = 8$
- Total amortized cost $= 2 + 2 + 2 + 0 + 0 + 2 + 0 = 8$
- Amortized cost per operation $= \frac{8}{7} \approx O(1)$

Conclusion: All three methods (aggregate, accounting, and potential) confirm that the amortized cost per stack operation is $O(1)$, even though individual operations like `multipop` can be expensive.

1.6 Asymptotic Notations

Complexity analysis becomes much easier if we analyze an algorithm in terms of *bounds* (upper and lower) rather than exact values. **Asymptotic notation** is a way of comparing functions that ignores constant factors and small input sizes. Three notations are used to express the running-time complexity of an algorithm.

Note:-

Why are asymptotic notations important?

- They give a simple characterization of an algorithm's efficiency.
- They allow the comparison of the performance of various algorithms.

1.6.1 Big-O (O) Notation – Upper Bound

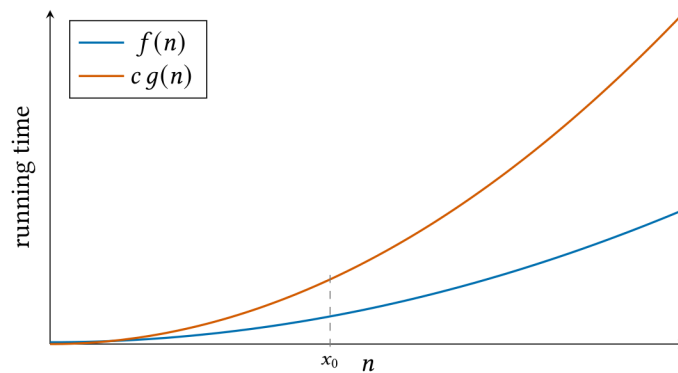
Definition 1.6.1

For two functions $f(x)$ and $g(x)$, we say $f(x) = O(g(x))$ ($f(x)$ is said to be big-oh of $g(x)$) if and only if there exist two positive constants c and x_0 such that

$$\text{for all } x \geq x_0, \quad f(x) \leq c g(x) \quad .$$

This says that $g(x)$ is an **upper bound** of $f(x)$.

Geometrical Interpretation: For all $n \geq x_0$, the plot of $f(n)$ lies on or below the curve $c g(n)$.



Example 1.6.1

Find the Big-O of $f(n) = 3n^2 + 4n + 7$

$$\begin{aligned} f(n) &= 3n^2 + 4n + 7 \leq 3n^2 + 4n^2 + 7n^2 \\ &= 14n^2 \quad (\text{for } n \geq 1) \end{aligned}$$

So $f(n) \leq 14n^2$. With $c = 14$ and $g(n) = n^2$, we get $f(n) = O(g(n)) = O(n^2)$.

1.6.2 Big-Omega(Ω) Notation – Lower Bound

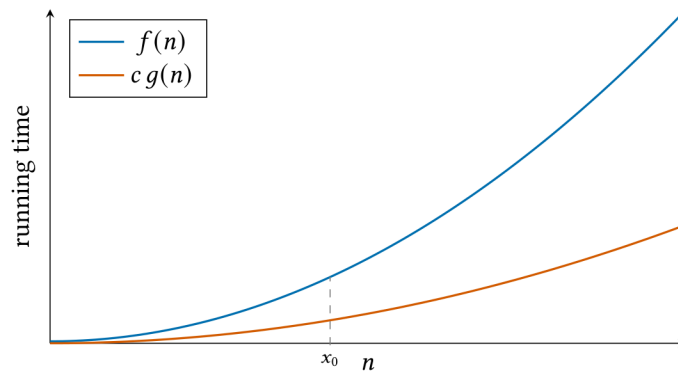
Definition 1.6.2

For two functions $f(x)$ and $g(x)$, we say $f(x) = \Omega(g(x))$ ($f(x)$ is said to be big-omega of $g(x)$) if and only if there exist two positive constants c and x_0 such that

$$\text{for all } x \geq x_0, \quad f(x) \geq c g(x) \quad .$$

This says that $g(x)$ is an **lower bound** of $f(x)$.

Geometrical Interpretation: For all $n \geq x_0$, the plot of $f(n)$ lies on or above the curve $c g(n)$.

**Example 1.6.2**

Find the Big- Ω of $f(n) = 3n^2 + 4n + 7$.

Since $f(n) = 3n^2 + 4n + 7 \geq 3n^2$ for all $n \geq 1$, we have $f(n) \geq 3n^2$. With $c = 3$ and $g(n) = n^2$, we get $f(n) = \Omega(g(n)) = \Omega(n^2)$.

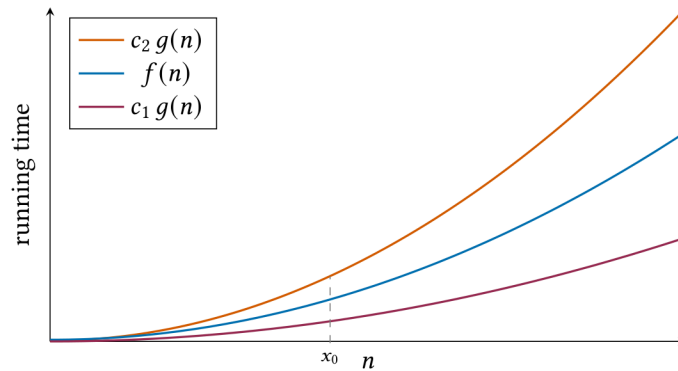
1.6.3 Big-Theta (Θ) Notation – Tight Bound**Definition 1.6.3**

For two functions $f(x)$ and $g(x)$, we say $f(x) = \Theta(g(x))$ ($f(x)$ is big-theta of $g(x)$) if and only if there exist three positive constants c_1 , c_2 , and x_0 such that

$$\text{for all } x \geq x_0 \quad c_1 g(x) \leq f(x) \leq c_2 g(x)$$

This gives an **asymptotically tight bound** of $f(x)$.

Geometrical Interpretation: For all $n \geq x_0$, the plot of $f(n)$ is sandwiched between $c_1 g(n)$ and $c_2 g(n)$.

**Example 1.6.3**

We show that $f(n) = 3n^2 + 4n + 7 = \Theta(n^2)$.

Lower bound

For all $n \geq 1$:

$$f(n) = 3n^2 + 4n + 7 \geq 3n^2.$$

Thus with $c_1 = 3$ and $n_0 = 1$ we have $f(n) \geq c_1 n^2$.

Upper bound

For $n \geq 1$, note that $n \leq n^2$ and $1 \leq n^2$. Hence

$$4n \leq 4n^2, \quad 7 \leq 7n^2.$$

Therefore

$$f(n) = 3n^2 + 4n + 7 \leq 3n^2 + 4n^2 + 7n^2 = 14n^2.$$

Take $c_2 = 14$ and the same $n_0 = 1$ to get $f(n) \leq c_2 n^2$.

Conclusion

For all $n \geq 1$:

$$3n^2 \leq f(n) \leq 14n^2,$$

which proves $f(n) = \Theta(n^2)$.

1.7 Concept of best case, average case and worst case performance of an algorithm

The running time of an algorithm often depends not only on the size of the input but also on the particular input instance. For a fixed input size n , different inputs may cause the algorithm to take different amounts of time. This leads to three standard ways of describing performance.

Example 1.7.1

Suppose an algorithm works on an array of 100 numbers. If the array is already sorted, the algorithm may finish quickly. If the array is in reverse order, it may take much longer.

1.7.1 Worst Case Performance

Definition 1.7.1

The **worst-case** running time is the *maximum* time taken by the algorithm over all inputs of size n . It gives an **upper bound** and is usually expressed using Big-O notation.

The worst case is the most commonly used measure because:

- It guarantees the algorithm will never take longer than this time.
- For many algorithms the worst case occurs fairly often.
- The average case is often as bad as the worst case.

1.7.2 Best Case Performance

Definition 1.7.2

The **best-case** running time is the *minimum* time taken by the algorithm over all inputs of size n . It gives a **lower bound** and is usually expressed using Big-Ω notation.

The best case is generally not very useful for guaranteeing performance, because it may correspond to a very rare input.

1.7.3 Average-Case Performance

Definition 1.7.3

The **average-case** running time is the *expected* time taken by the algorithm, averaged over all possible inputs of size n (assuming a probability distribution over the inputs).

The average case is often difficult to compute because it requires knowledge of the statistical distribution of inputs.

1.8 Modeling Algorithms by Recurrence Relation

A **recurrence relation** is an equation that expresses the running time $T(n)$ of a recursive algorithm in terms of its running time on smaller inputs. Each recursive call breaks the problem into smaller subproblems, so the total work is written as: the work done inside one call (non-recursive part) plus the cost of the recursive calls on smaller sizes.

A recurrence has two parts:

- **Base case:** cost when the input is smallest (e.g. $T(1) = c$), which stops the recursion.
- **Recursive case:** cost expressed in terms of smaller inputs (e.g. $T(n) = T(n-1) + c$).

1.8.1 Forming a Recurrence Relation

Example 1: Factorial (Linear Recursion)

```
1  int fact(int n) {
2      if (n <= 1)           // base case
3          return 1;
4      return n * fact(n-1);  // one recursive call on (n-1)
5  }
```

Each call does constant work c and makes one call of size $n-1$.

$$T(n) = T(n-1) + c, \quad T(1) = c$$

1.9 Solving recurrence relation for evaluating computational complexity

Definition 1.9.1

A **recurrence** is an equation or inequality that describes a function in terms of its values on smaller inputs. To *solve* a recurrence means to obtain a closed-form function (defined on the natural numbers) that satisfies the recurrence.

$$T(n) = \underbrace{\text{time for smaller subproblems}}_{\text{recursive part}} + \underbrace{\text{extra work}}_{\text{non-recursive part}}$$

1.9.1 Examples of Recurrence Models

(a) **Factorial:**

$$T(n) = \begin{cases} 1 & n = 1 \\ T(n-1) + O(1) & n > 1 \end{cases}$$

(b) n^{th} **Fibonacci number:**

$$T(n) = \begin{cases} 1 & n = 1 \\ 1 & n = 2 \\ T(n-1) + T(n-2) + O(1) & n > 2 \end{cases}$$

(c) Binary Search:

$$T(n) = \begin{cases} 1 & n = 1 \\ T(n/2) + O(1) & n > 1 \end{cases}$$

The process of finding the solution of a given recurrence relation in terms of Big-O notation is **solving recurrences**. Popular methods:

- Iteration Method (Backward Substitution)
- Recursion Tree Method
- Substitution Method
- Master Method

1.9.2 Iteration Method

In the iteration method we expand the given relation repeatedly until the boundary condition is met, so that a summation independent of n is obtained. It is also called the *iterative method* or *backward substitution*.

Example 1.9.1

Iteration Example 1: $T(n) = 2T(n/2) + 1$

$$\begin{aligned} T(n) &= 2T(n/2) + 1 \\ &= 2[2T(n/4) + 1] + 1 \\ &= 2^2T(n/4) + 2 + 1 \end{aligned}$$

Expand again:

$$\begin{aligned} T(n) &= 2^2[T(n/4)] + 2 + 1 \\ &= 2^2[2T(n/8) + 1] + 2 + 1 \\ &= 2^3T(n/8) + 2^2 + 2 + 1 \end{aligned}$$

Expand once more:

$$\begin{aligned} T(n) &= 2^3[2T(n/16) + 1] + 2^2 + 2 + 1 \\ &= 2^4T(n/16) + 2^3 + 2^2 + 2 + 1 \end{aligned}$$

Continuing in this way, after k expansions we get:

$$T(n) = 2^k T\left(\frac{n}{2^k}\right) + (2^{k-1} + 2^{k-2} + \dots + 2^2 + 2 + 1)$$

Find the stopping condition

The expansion stops when the subproblem size becomes 1.

Thus,

$$\frac{n}{2^k} = 1$$

Solving for k :

$$\begin{aligned} n &= 2^k \\ k &= \log_2 n \end{aligned}$$

We know the base condition:

$$T(1) = 1$$

$$T(n) = 2^k T(1) + (2^{k-1} + 2^{k-2} + \cdots + 2 + 1)$$

Since $T(1) = 1$:

$$T(n) = 2^k + (2^{k-1} + 2^{k-2} + \cdots + 2 + 1)$$

The summation is a geometric series:

$$1 + 2 + 2^2 + \cdots + 2^{k-1} = 2^k - 1$$

Therefore,

$$\begin{aligned} T(n) &= 2^k + (2^k - 1) \\ &= 2 \cdot 2^k - 1 \end{aligned}$$

Since $2^k = n$:

$$T(n) = 2n - 1$$

Hence,

$$T(n) = O(n)$$

Therefore, the recurrence grows linearly with input size.

Example 1.9.2

Iteration Example 2: Solve the recurrence

$$T(n) = T(n/3) + O(n)$$

Assume the non-recursive work is cn , where c is a constant. Then,

$$T(n) = T(n/3) + cn$$

Expand the recurrence by substituting $T(n/3)$:

$$\begin{aligned} T(n) &= T(n/3) + cn \\ &= [T(n/3^2) + c(n/3)] + cn \\ &= T(n/3^2) + \frac{cn}{3} + cn \end{aligned}$$

Expand again:

$$\begin{aligned} T(n) &= [T(n/3^3) + c(n/3^2)] + \frac{cn}{3} + cn \\ &= T(n/3^3) + \frac{cn}{3^2} + \frac{cn}{3} + cn \end{aligned}$$

Expand once more:

$$T(n) = T(n/3^4) + \frac{cn}{3^3} + \frac{cn}{3^2} + \frac{cn}{3} + cn$$

Continuing this process, after k expansions we obtain:

$$T(n) = T\left(\frac{n}{3^k}\right) + cn\left(1 + \frac{1}{3} + \frac{1}{3^2} + \cdots + \frac{1}{3^{k-1}}\right)$$

Determining the stopping condition.

The recursion stops when the subproblem size becomes 1. Thus,

$$\frac{n}{3^k} = 1$$

Solving for k :

$$\begin{aligned} n &= 3^k \\ k &= \log_3 n \end{aligned}$$

We know the base condition:

$$T(1) = 1$$

Then,

$$T(n) = 1 + cn\left(1 + \frac{1}{3} + \frac{1}{3^2} + \cdots + \frac{1}{3^{k-1}}\right)$$

The bracketed expression is a geometric series with:

$$a = 1, \quad r = \frac{1}{3}$$

The sum of an geometric series for $|r| < 1$ is:

$$\begin{aligned} \frac{a(1-r^k)}{1-r} \\ \frac{1 - \left(\frac{1}{3}\right)^k}{1 - \frac{1}{3}} &= \frac{1 - \frac{1}{3^k}}{\frac{2}{3}} = \frac{3}{2} \left(1 - \frac{1}{3^k}\right) \end{aligned}$$

Since $3^k = n$, we have:

$$T(n) = 1 + cn \cdot \frac{3}{2} \left(1 - \frac{1}{3^k}\right)$$

$$T(n) = 1 + \frac{3cn}{2} \left(1 - \frac{1}{n}\right)$$

$$T(n) = 1 + \frac{3c}{2}(n-1)$$

Ignoring constants in asymptotic notation:

$$\boxed{T(n) = O(n)}$$

Thus, the recurrence has linear time complexity.

Example 1.9.3

Iteration Example 3: Solve the recurrence

$$T(n) = T(n-1) + O(1)$$

Assume the constant work at each step is 1. Then,

$$T(n) = T(n-1) + 1$$

Expand the recurrence by substituting $T(n-1)$:

$$\begin{aligned} T(n) &= T(n-1) + 1 \\ &= T(n-2) + 1 + 1 \\ &= T(n-3) + 1 + 1 + 1 \\ &\vdots \\ &= T(n-k) + \underbrace{1 + 1 + \dots + 1}_{k \text{ times}} \end{aligned}$$

Determine stopping condition

The recursion stops when the problem size becomes 1:

$$n - k = 1$$

So,

$$k = n - 1$$

Substitute value of k

$$\begin{aligned} T(n) &= T(1) + k \\ &= T(1) + (n-1) \end{aligned}$$

The base case:

$$T(1) = 1$$

So,

$$\begin{aligned} T(n) &= 1 + (n-1) \\ &= n \end{aligned}$$

$$\boxed{T(n) = O(n)}$$

Thus, the recurrence grows linearly with input size.

Example 1.9.4

Iteration Example 4: Solve the recurrence

$$T(n) = 2T(n/2) + n$$

Expand the recurrence by substituting $T(n/2)$:

$$\begin{aligned}
T(n) &= 2T(n/2) + n \\
&= 2 \left[2T(n/4) + \frac{n}{2} \right] + n \\
&= 2^2 T(n/4) + n + n \\
&= 2^3 T(n/8) + n + n + n \\
&\vdots \\
&= 2^k T\left(\frac{n}{2^k}\right) + \underbrace{n + n + \cdots + n}_{k \text{ times}}
\end{aligned}$$

Determine stopping condition

The recursion stops when the subproblem size becomes 1:

$$\frac{n}{2^k} = 1$$

Solving:

$$\begin{aligned}
n &= 2^k \\
k &= \log_2 n
\end{aligned}$$

Substitute value of k

The base case:

$$T(1) = 1$$

Then,

$$T(n) = 2^k T(1) + kn$$

Since $2^k = n$:

$$T(n) = n + n \log n$$

$$\boxed{T(n) = O(n \log n)}$$

Example 1.9.5

Example 5: Solve the recurrence

$$T(n) = T(n/3) + n$$

Expand the recurrence by substituting $T(n/3)$:

$$\begin{aligned}
T(n) &= T(n/3) + n \\
&= T(n/3^2) + \frac{n}{3} + n \\
&= T(n/3^3) + \frac{n}{3^2} + \frac{n}{3} + n \\
&\vdots \\
&= T(n/3^k) + n \left(1 + \frac{1}{3} + \frac{1}{3^2} + \cdots + \frac{1}{3^{k-1}} \right)
\end{aligned}$$

Determine stopping condition

The recursion stops when the subproblem size becomes 1:

$$\frac{n}{3^k} = 1$$

Solving:

$$\begin{aligned}
n &= 3^k \\
k &= \log_3 n
\end{aligned}$$

Evaluate the geometric series

The bracketed term is a geometric series:

$$1 + \frac{1}{3} + \frac{1}{3^2} + \cdots + \frac{1}{3^{k-1}}$$

Using the formula:

$$S_k = \frac{a(1-r^k)}{1-r}, \quad a = 1, \quad r = \frac{1}{3}$$

$$\begin{aligned}
S_k &= \frac{1 \cdot (1 - (1/3)^k)}{1 - \frac{1}{3}} \\
&= \frac{1 - (1/3)^k}{2/3} \\
&= \frac{3}{2} \left(1 - (1/3)^k \right)
\end{aligned}$$

Since:

$$(1/3)^k = \frac{1}{3^k} = \frac{1}{n}$$

we get:

$$S_k = \frac{3}{2} \left(1 - \frac{1}{n} \right)$$

Substitute back into $T(n)$

$$T(n) = T(1) + n \cdot \frac{3}{2} \left(1 - \frac{1}{n} \right)$$

Assume:

$$T(1) = 1$$

So,

$$\begin{aligned} T(n) &= 1 + \frac{3n}{2} - \frac{3}{2} \\ &= \frac{3n}{2} - \frac{1}{2} \end{aligned}$$

$$\boxed{T(n) = O(n)}$$

Example 1.9.6

Iteration Example 6: Solve the recurrence

$$T(n) = 2T(n/2) + \frac{n}{\log n}$$

Expand the recurrence by substituting $T(n/2)$:

$$T(n) = 2T(n/2) + \frac{n}{\log n}$$

Substitute the recurrence for $T(n/2)$:

$$\begin{aligned} T(n) &= 2 \left[2T(n/4) + \frac{n/2}{\log(n/2)} \right] + \frac{n}{\log n} \\ &= 2^2 T(n/4) + \frac{n}{\log(n/2)} + \frac{n}{\log n} \end{aligned}$$

Expand again:

$$T(n) = 2^3 T(n/8) + \frac{n}{\log(n/4)} + \frac{n}{\log(n/2)} + \frac{n}{\log n}$$

Continuing this process, after k expansions:

$$T(n) = 2^k T(n/2^k) + \sum_{i=0}^{k-1} \frac{n}{\log(n/2^i)}$$

Determine the stopping condition.

The recursion stops when the subproblem size becomes 1:

$$\frac{n}{2^k} = 1$$

Thus,

$$\begin{aligned} n &= 2^k \\ k &= \log_2 n \end{aligned}$$

Simplifying

Using $\log_2(\cdot)$ throughout, since $n = 2^k$ we have

$$\log_2(n/2^i) = \log_2(2^k/2^i) = \log_2(2^{k-i}) = k - i$$

Therefore the summation becomes:

$$\sum_{i=0}^{k-1} \frac{n}{k-i} = n \sum_{i=0}^{k-1} \frac{1}{k-i}$$

Reversing the order of summation ($j = k - i$):

$$= n \left(\frac{1}{1} + \frac{1}{2} + \frac{1}{3} + \cdots + \frac{1}{k} \right) = n H_k$$

where H_k is the harmonic series:

$$H_k = 1 + \frac{1}{2} + \frac{1}{3} + \cdots + \frac{1}{k}.$$

We know:

$$H_k \approx \ln k.$$

Substituting back, and using $T(1) = O(1)$ with $2^k = n$:

$$T(n) = 2^k T\left(\frac{n}{2^k}\right) + \sum_{i=0}^{k-1} \frac{n}{\log(n/2^i)}$$

$$T(n) \approx n \cdot T(1) + n H_k \approx n \cdot 1 + n \ln k$$

$$T(n) \approx n \ln k.$$

Since:

$$k = \log_2 n,$$

we get:

$$T(n) \approx n \ln(\log_2 n).$$

Therefore,

$$\boxed{T(n) = O(n \log \log n)}$$

1.9.3 Recursion Tree Method

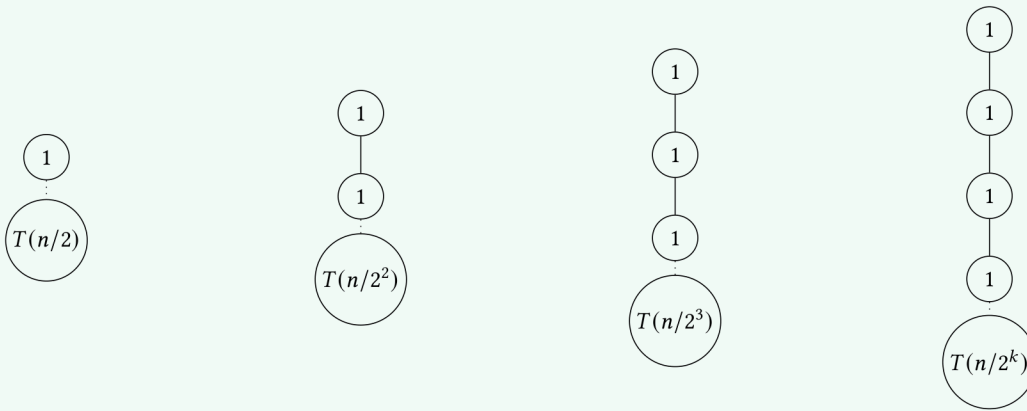
Definition 1.9.2

The **Recursion Tree Method** is a pictorial representation of the iteration method in the form of a tree, where at each level the nodes are expanded. Each node represents the cost of a single sub-problem. We sum the costs at each level to get the per-level cost, then sum all per-level costs to get the total cost. It is especially useful for divide-and-conquer algorithms.

Example 1.9.7

Solve the following recurrence relation using the recursion tree method.

$$T(n) = \begin{cases} 1 & n = 1 \\ T(n/2) + 1 & n > 1 \end{cases}$$



Cost at each level = 1. Assume $n/2^k = 1 \Rightarrow n = 2^k \Rightarrow k = \log n$.

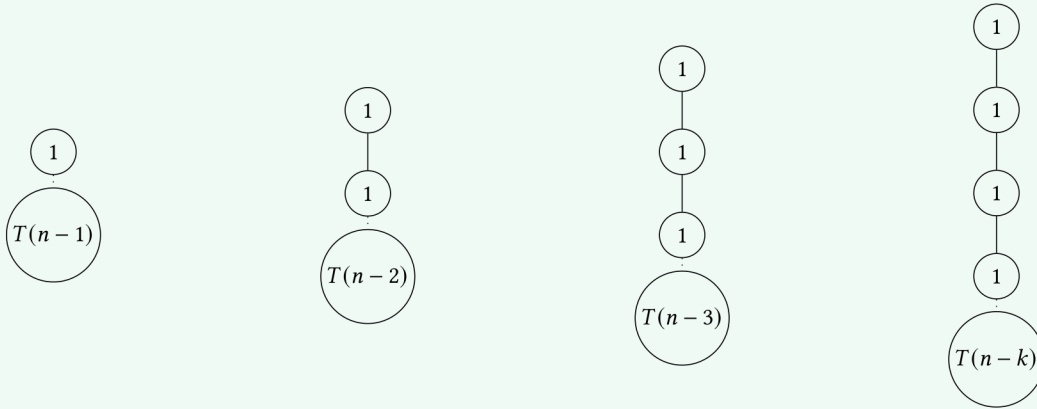
$$\begin{aligned} \text{Total} &= \underbrace{1 + 1 + \cdots + 1}_{k \text{ times}} + T(n/2^k) \\ &= \underbrace{1 + 1 + \cdots + 1}_{k \text{ times}} + T(1) \\ &= k + 1 = \log n + 1 = O(\log n). \end{aligned}$$

$$\boxed{T(n) = O(\log n)}$$

Example 1.9.8

Solve the following recurrence relation using the recursion tree method.

$$T(n) = \begin{cases} 1 & n = 1 \\ T(n-1) + 1 & n > 1 \end{cases}$$



Cost at each level = 1. Assume $n - k = 1 \Rightarrow k = n - 1$.

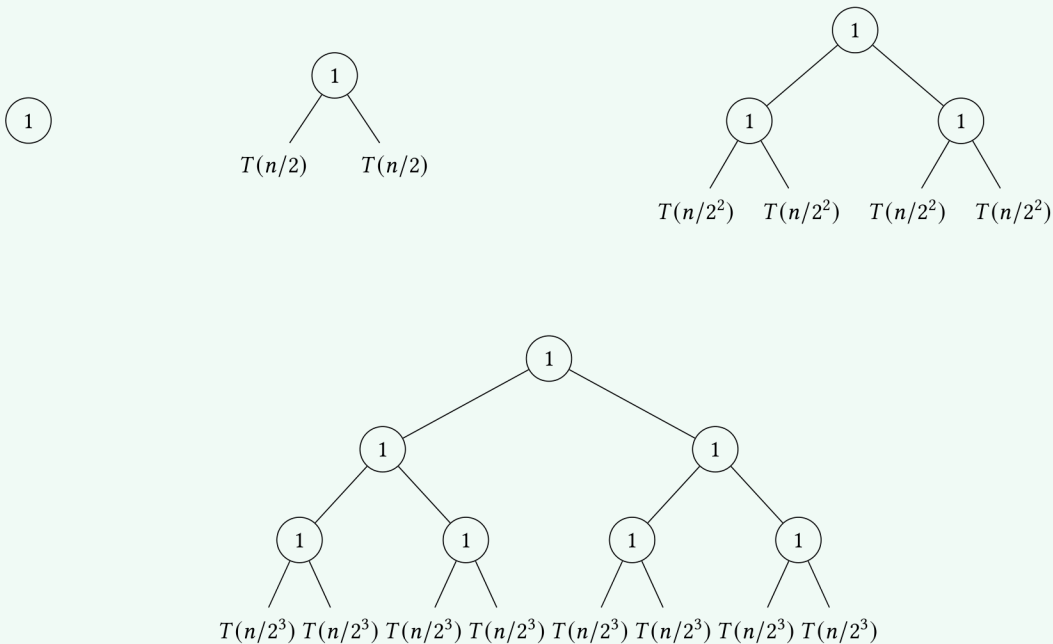
$$\begin{aligned}
 \text{Total} &= \underbrace{1 + 1 + \cdots + 1}_{k \text{ times}} + T(n - k) \\
 &= \underbrace{1 + 1 + \cdots + 1}_{k \text{ times}} + T(1) \\
 &= k + 1 = n - 1 + 1 = n = O(n).
 \end{aligned}$$

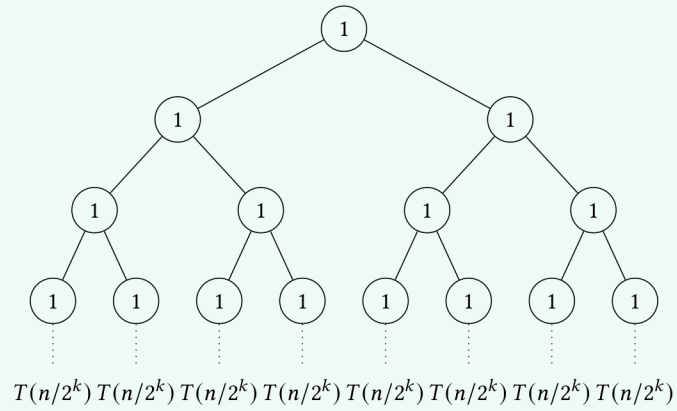
$$\boxed{T(n) = O(n)}$$

Example 1.9.9

Solve the following recurrence relation using the recursion tree method.

$$T(n) = \begin{cases} 1 & \text{if } n = 1, \\ 2T(n/2) + 1 & \text{if } n > 1. \end{cases}$$





Now

$$\begin{aligned} T(n) &= 2^0 + 2^1 + 2^2 + \cdots + 2^{k-1} + 2^k T(n/2^k) \\ &= 2^0 + 2^1 + 2^2 + \cdots + 2^{k-1} + 2^k \cdot 1 \end{aligned}$$

Using the formula for a geometric series,

$$S = \frac{a(r^k - 1)}{r - 1} \text{ where } a = 1 \text{ and } r = 2:$$

$$\begin{aligned} T(n) &= \frac{1 \cdot (2^{k+1} - 1)}{2 - 1} \\ &= \frac{2^{k+1} - 1}{1} \\ &= 2^{k+1} - 1 \\ &= 2 \cdot 2^k - 1 \end{aligned}$$

Putting $n/2^k = 1$:

$$\Rightarrow n = 2^k$$

Taking log on both sides:

$$\begin{aligned} \Rightarrow \log n &= \log 2^k \\ \Rightarrow \log n &= k \log 2 \\ \Rightarrow k \log 2 &= \log n \\ \Rightarrow k &= \log_2 n \end{aligned}$$

Substitute $2^k = n$ back into $T(n)$:

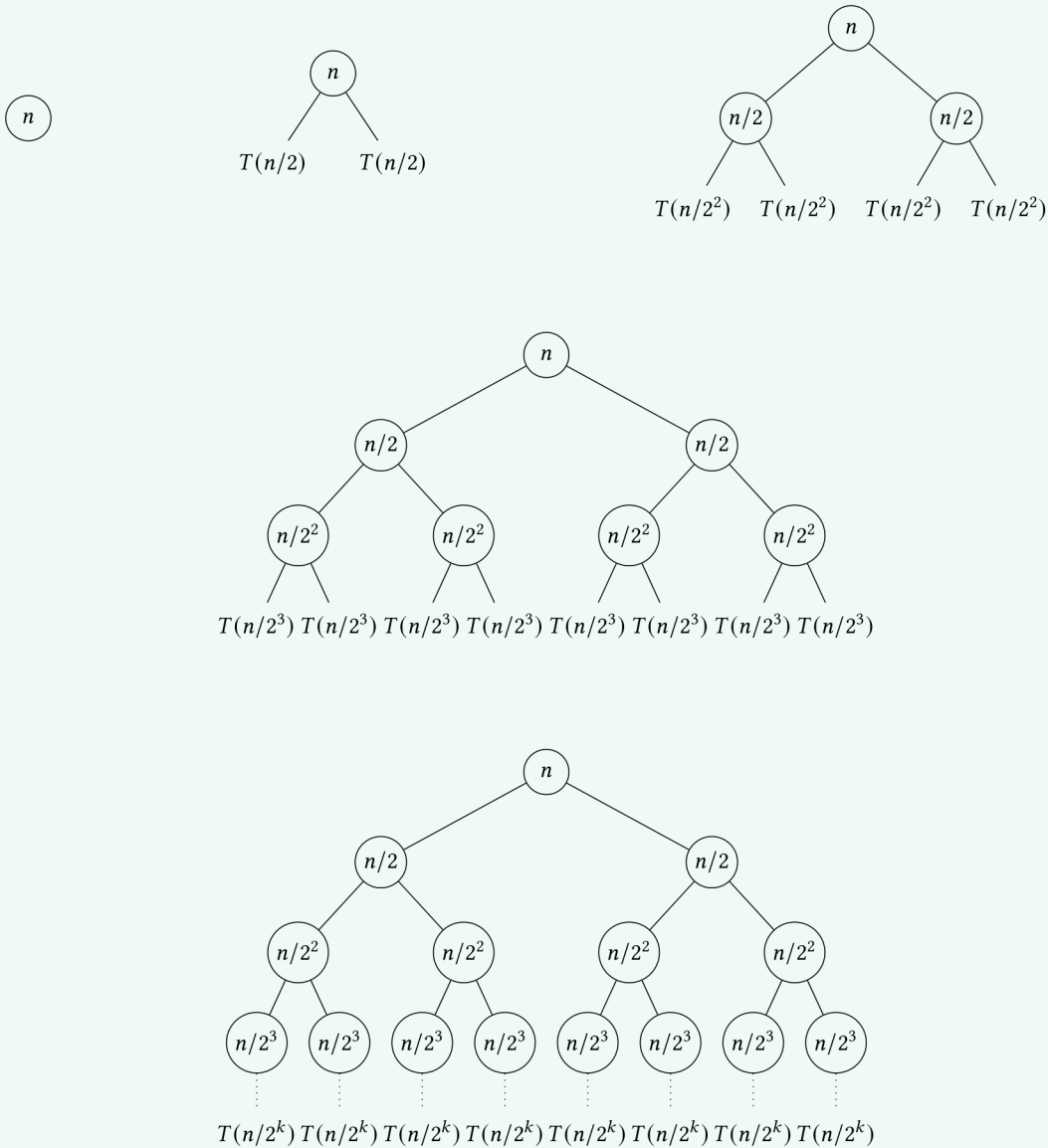
$$\begin{aligned} T(n) &= 2 \cdot 2^k - 1 \\ &= 2n - 1 \end{aligned}$$

$T(n) = O(n)$

Example 1.9.10

Solve the following recurrence relation using the recursion tree method.

$$T(n) = \begin{cases} 1 & \text{if } n = 1, \\ 2T(n/2) + n & \text{if } n > 1. \end{cases}$$



Now

$$\begin{aligned} T(n) &= \underbrace{n + n + n + \cdots + n}_{k \text{ times}} + 2^k T\left(\frac{n}{2^k}\right) \\ &= nk + 2^k T\left(\frac{n}{2^k}\right) \end{aligned}$$

Putting $n/2^k = 1$:

$$\Rightarrow n = 2^k$$

Taking log on both sides:

$$\Rightarrow \log n = \log 2^k$$

$$\Rightarrow \log n = k \log 2$$

$$\Rightarrow k \log 2 = \log n$$

$$\Rightarrow k = \log_2 n$$

Substitute $k = \log_2 n$ and $T(1) = 1$:

$$\begin{aligned} T(n) &= nk + nT(1) \\ &= n(\log_2 n) + n \cdot 1 \\ &= n \log_2 n + n \end{aligned}$$

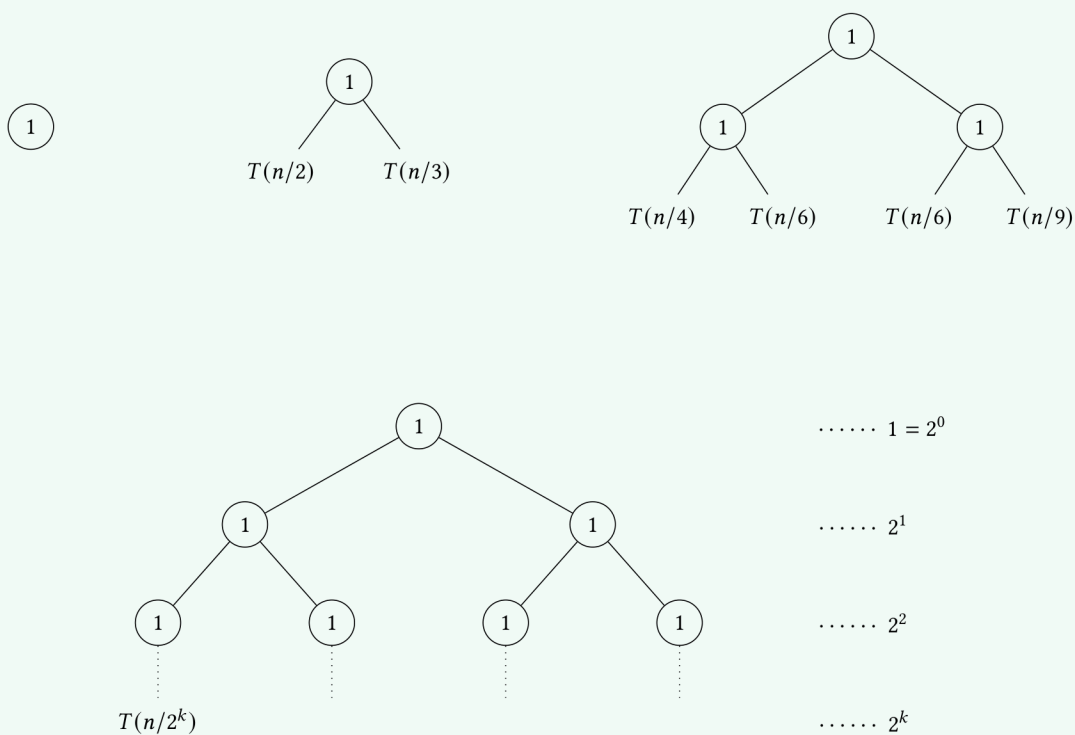
Hence,

$$T(n) = O(n \log n)$$

Example 1.9.11

Solve the following recurrence relation using the recursion tree method.

$$\begin{cases} T(1) = 1 & \text{when } n = 1 \\ T(n) = T(n/2) + T(n/3) + O(1) & \text{when } n > 1 \end{cases}$$



Now total cost,

$$\begin{aligned}
 T(n) &\leq 1 + 2^1 + 2^2 + \dots + 2^k && \text{refer to note below} \\
 &= 1 + \left[2(2^k - 1) / (2 - 1) \right] \\
 &= 1 + 2 \cdot 2^k - 2 \\
 &= 2 \cdot 2^k - 1
 \end{aligned}$$

Note: The formula assumes *nobody dies early*. But in a real lopsided tree, the small $n/3$ branches die early, so the formula counts *more* nodes than really exist — which is exactly why the relation is “ $\leq$ ” and not “ $=$ ”.

Let us put $n/2^k = 1$:

$$\Rightarrow n = 2^k$$

Taking log on both sides:

$$\begin{aligned}
 \Rightarrow \log n &= \log 2^k \\
 \Rightarrow k \log 2 &= \log n \\
 \Rightarrow k &= \log n
 \end{aligned}$$

Now

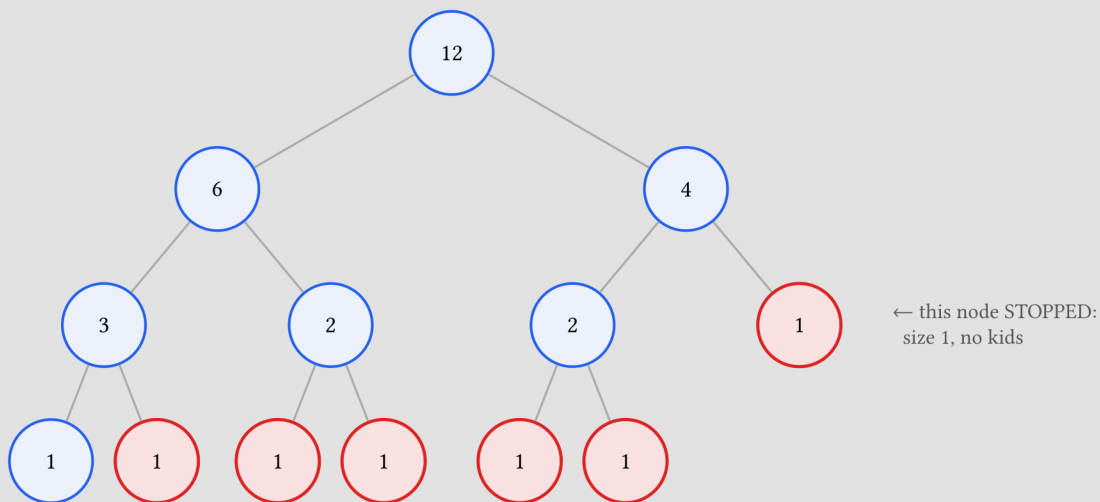
$$\begin{aligned}
 T(n) &= 2 \cdot 2^k - 1 \\
 &= 2n - 1
 \end{aligned}$$

Hence, $T(n) = O(n)$.

Note:-

Concrete recursion tree for $n = 12$

Rule: every node splits into a $\lfloor n/2 \rfloor$ child and a $\lfloor n/3 \rfloor$ child. A node of size 1 STOPS (no children).



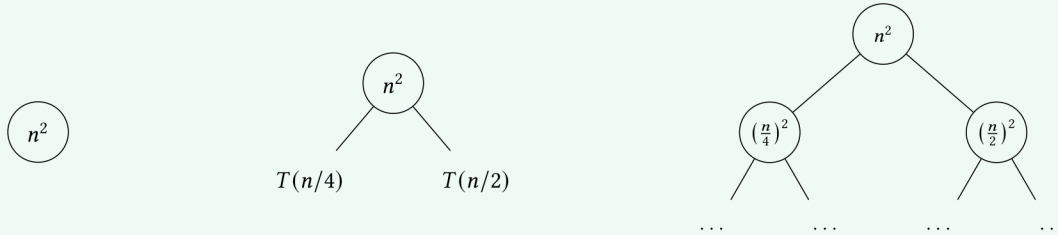
The red 1 nodes are **dead ends** – they make NO children.
So the real tree has *fewer* nodes than a full doubling tree $(1 + 2 + 4 + 8 + \dots)$.

Example 1.9.12

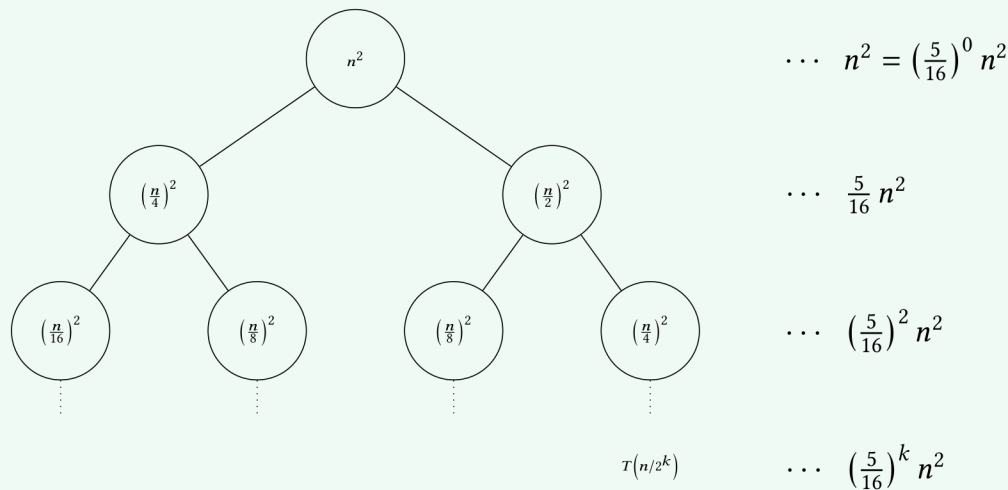
Solve the following recurrence relation using the recursion tree method.

$$\begin{cases} T(1) = 1 & \text{when } n = 1 \\ T(n) = T(n/4) + T(n/2) + n^2 & \text{when } n > 1 \end{cases}$$

Solution: At each node we do n^2 work. A node of size n splits into a child of size $n/4$ and a child of size $n/2$.



Cost per level. Add the work on each level:



Why 5/16: the two children contribute $\left(\frac{n}{4}\right)^2 + \left(\frac{n}{2}\right)^2 = \frac{n^2}{16} + \frac{n^2}{4} = \frac{5}{16} n^2$. So each level is $\frac{5}{16}$ times the level above it.

Now total cost. This is a geometric series with first term $a = 1$ and ratio $r = \frac{5}{16}$. Using $S = \frac{a(1-r^k)}{1-r}$:

$$\begin{aligned} T(n) &= n^2 \left[\left(\frac{5}{16}\right)^0 + \left(\frac{5}{16}\right)^1 + \left(\frac{5}{16}\right)^2 + \cdots + \left(\frac{5}{16}\right)^k \right] && \text{(refer to note)} \\ &= n^2 \cdot \frac{1 - \left(\frac{5}{16}\right)^{k+1}}{1 - \frac{5}{16}} \\ &= n^2 \cdot \frac{16}{11} \left[1 - \left(\frac{5}{16}\right)^{k+1} \right] \\ &\leq \frac{16}{11} n^2 && \text{since } \left(\frac{5}{16}\right)^{k+1} > 0, \text{ so the bracket } < 1. \end{aligned}$$

The tree stops when the subproblem reaches size 1. On the longest path (dividing by 2 each time) put $n/2^k = 1$:

$$\Rightarrow n = 2^k$$

Taking log on both sides:

$$\Rightarrow \log n = \log 2^k$$

$$\Rightarrow \log n = k \log 2$$

$$\Rightarrow k = \log_2 n$$

Now

$$\begin{aligned} T(n) &\leq \frac{16}{11} n^2 \\ &= c n^2 \quad (\text{a constant times } n^2). \end{aligned}$$

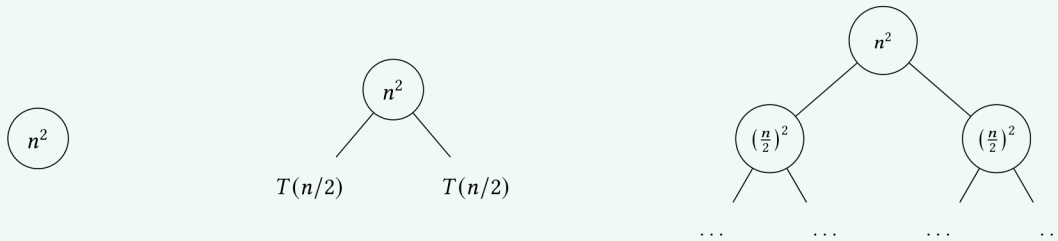
$$\text{Hence, } T(n) = O(n^2).$$

Example 1.9.13

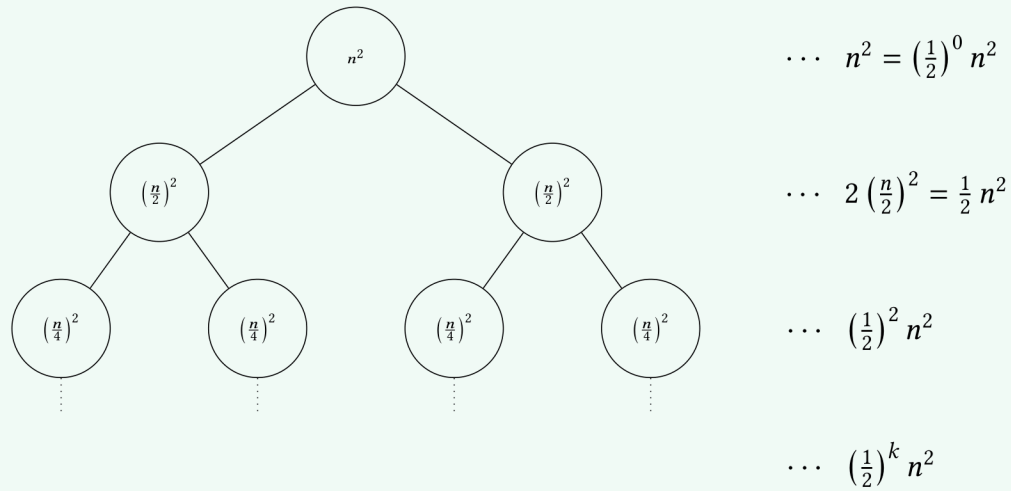
Solve the following recurrence relation using the recursion tree method.

$$\begin{cases} T(1) = 1 & \text{when } n = 1 \\ T(n) = 2T(n/2) + n^2 & \text{when } n > 1 \end{cases}$$

Solution: At each node we do n^2 work. A node of size n splits into *two* children, each of size $n/2$.



Cost per level. Add the work on each level:



Now total cost. This is a geometric series with first term $a = 1$ and ratio $r = \frac{1}{2}$. Using $S = \frac{a(1-r^k)}{1-r}$:

$$\begin{aligned}
 T(n) &= n^2 \left[\left(\frac{1}{2}\right)^0 + \left(\frac{1}{2}\right)^1 + \left(\frac{1}{2}\right)^2 + \dots + \left(\frac{1}{2}\right)^k \right] && \text{(refer to note)} \\
 &= n^2 \cdot \frac{1 - \left(\frac{1}{2}\right)^{k+1}}{1 - \frac{1}{2}} \\
 &= n^2 \cdot 2 \left[1 - \left(\frac{1}{2}\right)^{k+1} \right] \\
 &\leq 2 n^2 \quad \text{since } \left(\frac{1}{2}\right)^{k+1} > 0, \text{ so the bracket } < 1.
 \end{aligned}$$

The tree stops when the subproblem reaches size 1. Put $n/2^k = 1$:

$$\Rightarrow n = 2^k$$

Taking log on both sides:

$$\begin{aligned}
 \Rightarrow \log n &= \log 2^k \\
 \Rightarrow \log n &= k \log 2 \\
 \Rightarrow k &= \log n
 \end{aligned}$$

Now

$$\begin{aligned}
 T(n) &\leq 2 n^2 \\
 &= c n^2 \quad (\text{a constant times } n^2). \\
 \text{Hence, } T(n) &= O(n^2).
 \end{aligned}$$

1.9.4 Substitution Method

Definition 1.9.3

The substitution method has two steps:

1. guess the form of the solution,
2. use mathematical induction to show the guess is valid.

It is powerful for non-trivial recurrences and can establish both upper and lower bounds, but it only works when the solution can be guessed.

Example 1.9.14

Example 1: Show that $T(n) = O(n^3)$ is the complexity for the following recurrence relation using the substitution method.

$$\begin{cases} T(1) = 1 & \text{when } n = 1 \\ T(n) = 4T(n/2) + n & \text{when } n > 1 \end{cases}$$

Solution: We guess that $T(n) \leq cn^3$ for some constant $c > 0$ and all $n \geq n_0$, then prove it by mathematical induction.

Base step ($n = 1$):

$$T(1) = 1 \leq c \cdot 1^3 = c,$$

which is true for any constant $c \geq 1$.

Inductive step: Assume the bound holds for all $k < n$;

$$T(k) = ck^3$$

It is also true for $k = n/2$:

$$\begin{aligned} T(n/2) &\leq c \left(\frac{n}{2}\right)^3 \\ T(n/2) &= \frac{cn^3}{8} \end{aligned}$$

Then

$$\begin{aligned} T(n) &= 4T(n/2) + n \\ &\leq 4 \cdot \frac{cn^3}{8} + n \\ &= \frac{cn^3}{2} + n \\ &= cn^3 - \frac{cn^3}{2} + n \\ &= cn^3 - \left(\frac{cn^3}{2} - n\right) \\ &= cn^3 - n\left(\frac{cn^2}{2} - 1\right) \\ &\leq cn^3 \end{aligned}$$

for all $n > 0$ (the subtracted term $n\left(\frac{cn^2}{2} - 1\right)$ is non-negative when $c \geq 2$).

Hence $T(n) \leq cn^3$, i.e. $T(n) = O(n^3)$. **Proved** ■

Example 1.9.15

Example 2: Show that $O(n^2)$ is the solution of the following recurrence relation using the substitution method.

$$\begin{cases} T(n) = 1 & n = 1 \\ T(n) = 4T(n/2) + n & n > 1 \end{cases}$$

Solution:

Guess: $T(n) = O(n^2)$.

$$T(n) \leq c n^2 \quad \text{for some constant } c > 0 \text{ and } \forall n > n_0 \quad (1)$$

Now prove this relation by using mathematical induction.

Base step: For $n = 1$,

$$\begin{aligned} T(n) &= c \cdot 1^2 \\ &= 1 \leq c \quad \text{which is true for all +ve values of } c. \end{aligned}$$

Inductive step: Let us assume that it is true $\forall k < n$.

$$\text{Then } T(k) \leq c k^2 \quad (2)$$

It is also true for $k = n/2$. Now equation (2) becomes,

$$\begin{aligned} T(n/2) &\leq c (n/2)^2 \\ &= \frac{c n^2}{4} \end{aligned}$$

Now,

$$\begin{aligned} T(n) &= 4T(n/2) + n \\ &\leq 4 \cdot \frac{c n^2}{4} + n \\ &= c n^2 + n \\ \Rightarrow T(n) &= c n^2 + n \end{aligned}$$

It is not possible to show that $c n^2 + n \leq c n^2 \quad \forall n > 0$; thus we try to subtract a lower-order term as,

$$\Rightarrow T(n) \leq c n^2 - d n \quad [\text{since } c n^2 - d n \leq c n^2] \quad (3)$$

where c and d are +ve constants.

Now prove this relation by using mathematical induction.

Base step: For $n = 1$,

$$\begin{aligned} T(n) &= c \cdot 1^2 - d \cdot 1 \quad (\text{Definition}) \\ 1 &\leq c - d \quad \text{which is true for all +ve values of } c \text{ and } d \text{ with } d < c. \end{aligned}$$

Inductive step: Let us assume that it is true $\forall k < n$.

$$\text{Then } T(k) \leq c k^2 - d k \quad (4)$$

It is also true for $k = n/2$. Now equation (4) becomes,

$$T(n/2) \leq c (n/2)^2 - d n/2$$

$$= \frac{c n^2}{4} - \frac{d n}{2}$$

Now,

$$\begin{aligned} T(n) &= 4T(n/2) + n \\ &\leq 4 \left[\frac{c n^2}{4} - \frac{d n}{4} \right] + n \\ &\leq c n^2 - 2d n + n \\ &\leq c n^2 - d n - d n + n \\ &\leq (c n^2 - d n) - n(d - 1) \leq (c n^2 - d n) \\ \Rightarrow T(n) &\leq (c n^2 - d n) \quad \forall n > 0 \end{aligned}$$

Hence $T(n) = O(n^2)$. **Proved**

Example 1.9.16

Example 3: Show that $O(n^3)$ is the solution of the following recurrence relation using the substitution method.

$$T(n) = 8T(n/2) + n^2$$

Solution:

Let us guess, $T(n) = O(n^3)$.

$$T(n) \leq c n^3 \quad \text{for } \forall n > n_0 \quad (1)$$

Now prove this relation by using mathematical induction.

Base step: For $n = 1$,

$$\begin{aligned} T(n) &= c \cdot 1^3 \\ &= 1 \leq c \quad \text{which is true for all +ve values of } c. \end{aligned}$$

Inductive step: Let us assume that it is true $\forall k < n$.

$$\text{Then } T(k) \leq c k^3 \quad (2)$$

It is also true for $k = n/2$. Now equation (2) becomes,

$$\begin{aligned} T(n/2) &\leq c (n/2)^3 \\ &= \frac{c n^3}{8} \end{aligned}$$

Now,

$$\begin{aligned} T(n) &= 8T(n/2) + n^2 \\ &\leq 8 \cdot \frac{c n^3}{8} + n^2 \\ &= c n^3 + n^2 \\ \Rightarrow T(n) &= c n^3 + n^2 \end{aligned}$$

It is not possible to show that $c n^3 + n^2 \leq c n^3 \quad \forall n > 0$; thus we try to subtract a lower-order term as,

$$\Rightarrow T(n) \leq c n^3 - d n^2 \quad [\text{since } c n^3 - d n^2 \leq c n^3] \quad (3)$$

where c and d are +ve constants.

Now prove this relation by using mathematical induction.

Base step: For $n = 1$,

$$T(n) = c \cdot 1^3 - d \cdot 1$$

$$1 \leq c - d \quad \text{which is true for all +ve values of } c \text{ and } d \text{ with } d < c.$$

Inductive step: Let us assume that it is true $\forall k < n$.

$$\text{Then } T(k) \leq c k^3 - d k^2 \quad (4)$$

It is also true for $k = n/2$. Now equation (4) becomes,

$$\begin{aligned} T(n/2) &\leq c (n/2)^3 - d (n/2)^2 \\ &= \frac{c n^3}{8} - \frac{d n^2}{4} \end{aligned}$$

Now,

$$\begin{aligned} T(n) &= 8 T(n/2) + n^2 \\ &\leq 8 \left[\frac{c n^3}{8} - \frac{d n^2}{4} \right] + n^2 \\ &\leq c n^3 - 2d n^2 + n^2 \\ &\leq c n^3 - d n^2 - d n^2 + n^2 \\ &\leq (c n^3 - d n^2) - n^2 (d - 1) \leq (c n^3 - d n^2) \\ \Rightarrow T(n) &\leq (c n^3 - d n^2) \quad \forall n > 0 \end{aligned}$$

Thus $T(n) = O(n^3)$. **Proved**

1.9.5 Using Masters Theorem

Definition 1.9.4

The **Master Method** provides a direct solution for recurrences of the form

$$T(n) = a T(n/b) + f(n),$$

where $a \geq 1$, $b > 1$ are constants and $f(n)$ is an asymptotically positive function. Compare $f(n)$ with $n^{\log_b a}$.

Note:-

- **Case 1:** If $f(n) = O(n^{\log_b a - \varepsilon})$ for some $\varepsilon > 0$, then $T(n) = \Theta(n^{\log_b a})$.
- **Case 2:** If $f(n) = \Theta(n^{\log_b a})$, then $T(n) = \Theta(n^{\log_b a} \log n)$.
- **Case 3:** If $f(n) = \Omega(n^{\log_b a + \varepsilon})$ for some $\varepsilon > 0$ (and the regularity condition holds), then $T(n) = \Theta(f(n))$.
- **Case 4:** The Master Method *cannot* be applied (the gap between $f(n)$ and $n^{\log_b a}$ is not polynomial, or a is not constant).

Example 1.9.17**Master Example 1:** Solve the recurrence using the Master Theorem.

$$T(n) = 3T\left(\frac{n}{2}\right) + n$$

Step 1: Identify a , b , and $f(n)$.Compare $T(n) = 3T(n/2) + n$ with the standard form $T(n) = aT(n/b) + f(n)$:

$$a = 3, \quad b = 2, \quad f(n) = n.$$

Step 2: Compute the critical exponent $\log_b a$.

$$\log_b a = \log_2 3.$$

Numerically,

$$\log_2 3 = \frac{\ln 3}{\ln 2} = \frac{1.0986}{0.6931} \approx 1.585.$$

So the critical function is

$$n^{\log_b a} = n^{\log_2 3} \approx n^{1.585}.$$

Step 3: Compare $f(n)$ with $n^{\log_b a}$.Here $f(n) = n = n^1$. We compare the exponents:

$$\underbrace{1}_{\text{exponent of } f(n)} < \underbrace{1.585}_{\log_2 3}.$$

Since the exponent of $f(n)$ is *strictly smaller* than $\log_2 3$, $f(n)$ grows polynomially *slower* than the critical function. This points to **Case 1**.

Step 4: Verify the Case 1 condition formally.We must find an $\varepsilon > 0$ such that

$$f(n) = O\left(n^{\log_2 3 - \varepsilon}\right).$$

Take, for example, $\varepsilon = 0.1$. Then

$$n^{\log_2 3 - \varepsilon} = n^{1.585 - 0.1} = n^{1.485}.$$

Since

$$f(n) = n^1 \leq n^{1.485} \quad \text{for all } n \geq 1,$$

we have $f(n) = O(n^{1.085}) = O(n^{\log_2 3 - \varepsilon})$. The Case 1 condition holds. (Any ε with $0 < \varepsilon \leq \log_2 3 - 1 \approx 0.585$ works.)

Step 5: Apply Case 1 to get the answer.

By Case 1 of the Master Theorem,

$$T(n) = \Theta\left(n^{\log_b a}\right) = \Theta\left(n^{\log_2 3}\right) \approx \Theta\left(n^{1.585}\right).$$

$$T(n) = \Theta\left(n^{\log_2 3}\right) \approx \Theta\left(n^{1.585}\right)$$

Example 1.9.18**Master Example 2:** Solve the recurrence using the Master Theorem.

$$T(n) = 4T\left(\frac{n}{2}\right) + n^2$$

Step 1: Identify a , b , and $f(n)$. Compare $T(n) = 4T(n/2) + n^2$ with the standard form $T(n) = aT(n/b) + f(n)$:

$$a = 4, \quad b = 2, \quad f(n) = n^2.$$

Step 2: Compute the critical exponent $\log_b a$.

$$\log_b a = \log_2 4 = 2.$$

So the critical function is

$$n^{\log_b a} = n^{\log_2 4} = n^2.$$

Step 3: Compare $f(n)$ with $n^{\log_b a}$. Here $f(n) = n^2$. We compare the exponents:

$$\underbrace{2}_{\text{exponent of } f(n)} = \underbrace{2}_{\log_2 4}.$$

The exponent of $f(n)$ is *equal* to $\log_2 4$, so $f(n)$ grows at the *same* polynomial rate as the critical function. This points to **Case 2**.

Step 4: Verify the Case 2 condition formally. We must show

$$f(n) = \Theta(n^{\log_b a}) = \Theta(n^2).$$

Since $f(n) = n^2$ exactly, we trivially have

$$n^2 \leq f(n) \leq n^2 \quad \text{for all } n \geq 1,$$

so $f(n) = \Theta(n^2)$. The Case 2 condition holds.

Step 5: Apply Case 2 to get the answer. By Case 2 of the Master Theorem,

$$T(n) = \Theta(n^{\log_b a} \log n) = \Theta(n^2 \log n).$$

$$T(n) = \Theta(n^2 \log n)$$

Example 1.9.19

Master Example 3: Solve the recurrence using the Master Theorem.

$$T(n) = 9T\left(\frac{n}{3}\right) + n$$

Step 1: Identify a , b , and $f(n)$. Compare $T(n) = 9T(n/3) + n$ with the standard form $T(n) = aT(n/b) + f(n)$:

$$a = 9, \quad b = 3, \quad f(n) = n.$$

Step 2: Compute the critical exponent $\log_b a$.

$$\log_b a = \log_3 9 = 2.$$

So the critical function is

$$n^{\log_b a} = n^{\log_3 9} = n^2.$$

Step 3: Compare $f(n)$ with $n^{\log_b a}$. Here $f(n) = n = n^1$. We compare the exponents:

$$\underbrace{1}_{\text{exponent of } f(n)} < \underbrace{2}_{\log_3 9}.$$

Since the exponent of $f(n)$ is *strictly smaller* than $\log_3 9$, $f(n)$ grows polynomially *slower* than the critical function. This points to **Case 1**.

Step 4: Verify the Case 1 condition formally. We must find an $\varepsilon > 0$ such that

$$f(n) = O\left(n^{\log_3 9 - \varepsilon}\right).$$

Take, for example, $\varepsilon = 0.5$. Then

$$n^{\log_3 9 - \varepsilon} = n^{2 - 0.5} = n^{1.5}.$$

Since

$$f(n) = n^1 \leq n^{1.5} \quad \text{for all } n \geq 1,$$

we have $f(n) = O(n^{1.5}) = O(n^{\log_3 9 - \varepsilon})$. The Case 1 condition holds. (Any ε with $0 < \varepsilon \leq \log_3 9 - 1 = 1$ works.)

Step 5: Apply Case 1 to get the answer. By Case 1 of the Master Theorem,

$$T(n) = \Theta\left(n^{\log_b a}\right) = \Theta\left(n^{\log_3 9}\right) = \Theta(n^2).$$

$$T(n) = \Theta(n^2)$$

Example 1.9.20

Master Example 4: Solve the recurrence using the Master Theorem.

$$T(n) = 3T\left(\frac{n}{4}\right) + n \log n$$

Step 1: Identify a , b , and $f(n)$. Compare $T(n) = 3T(n/4) + n \log n$ with the standard form $T(n) = aT(n/b) + f(n)$:

$$a = 3, \quad b = 4, \quad f(n) = n \log n.$$

Step 2: Compute the critical exponent $\log_b a$.

$$\log_b a = \log_4 3.$$

Numerically,

$$\log_4 3 = \frac{\ln 3}{\ln 4} = \frac{1.0986}{1.3863} \approx 0.792.$$

So the critical function is

$$n^{\log_b a} = n^{\log_4 3} \approx n^{0.792}.$$

Step 3: Compare $f(n)$ with $n^{\log_b a}$. Here $f(n) = n \log n$, which grows like n^1 up to a logarithmic factor. We compare the exponents:

$$\underbrace{1}_{\text{exponent of } f(n)} > \underbrace{0.792}_{\log_4 3}.$$

Since the exponent of $f(n)$ is *strictly larger* than $\log_4 3$, $f(n)$ grows polynomially *faster* than the critical function. This points to **Case 3**.

Step 4: Verify the Case 3 conditions formally. (a) *Polynomial separation.* We must find an $\varepsilon > 0$ such that

$$f(n) = \Omega\left(n^{\log_4 3 + \varepsilon}\right).$$

Take, for example, $\varepsilon = 0.2$. Then $\log_4 3 + \varepsilon \approx 0.992 < 1$, and

$$f(n) = n \log n \geq n^{0.992} \quad \text{for all } n \geq 2,$$

so $f(n) = \Omega(n^{\log_4 3 + \varepsilon})$. (Any ε with $0 < \varepsilon \leq 1 - \log_4 3 \approx 0.208$ works, since $n \log n = \Omega(n)$.)

(b) *Regularity condition.* We must show $a f(n/b) \leq c f(n)$ for some constant $c < 1$ and all large n :

$$a f\left(\frac{n}{b}\right) = 3 \cdot \frac{n}{4} \log \frac{n}{4} \leq \frac{3}{4} n \log n = c f(n),$$

with $c = \frac{3}{4} < 1$ (using $\log(n/4) \leq \log n$). The regularity condition holds.

Step 5: Apply Case 3 to get the answer. By Case 3 of the Master Theorem,

$$T(n) = \Theta(f(n)) = \Theta(n \log n).$$

$$T(n) = \Theta(n \log n)$$

Example 1.9.21

Master Example 5: Solve the recurrence using the Master Theorem.

$$T(n) = 2 T\left(\frac{n}{4}\right) + n^{0.5}$$

Step 1: Identify a , b , and $f(n)$. Compare $T(n) = 2 T(n/4) + n^{0.5}$ with the standard form $T(n) = a T(n/b) + f(n)$:

$$a = 2, \quad b = 4, \quad f(n) = n^{0.5} = \sqrt{n}.$$

Step 2: Compute the critical exponent $\log_b a$.

$$\log_b a = \log_4 2 = \frac{1}{2} = 0.5.$$

So the critical function is

$$n^{\log_b a} = n^{\log_4 2} = n^{0.5} = \sqrt{n}.$$

Step 3: Compare $f(n)$ with $n^{\log_b a}$. Here $f(n) = n^{0.5}$. We compare the exponents:

$$\underbrace{0.5}_{\text{exponent of } f(n)} = \underbrace{0.5}_{\log_4 2}.$$

The exponent of $f(n)$ is *equal* to $\log_4 2$, so $f(n)$ grows at the *same* polynomial rate as the critical function. This points to **Case 2**.

Step 4: Verify the Case 2 condition formally. We must show

$$f(n) = \Theta\left(n^{\log_b a}\right) = \Theta\left(n^{0.5}\right).$$

Since $f(n) = n^{0.5}$ exactly, we trivially have

$$n^{0.5} \leq f(n) \leq n^{0.5} \quad \text{for all } n \geq 1,$$

so $f(n) = \Theta(n^{0.5})$. The Case 2 condition holds.

Step 5: Apply Case 2 to get the answer. By Case 2 of the Master Theorem,

$$T(n) = \Theta\left(n^{\log_b a} \log n\right) = \Theta\left(n^{0.5} \log n\right) = \Theta\left(\sqrt{n} \log n\right).$$

$$T(n) = \Theta\left(\sqrt{n} \log n\right)$$

Example 1.9.22**Master Example 6:** Solve the recurrence using the Master Theorem.

$$T(n) = 2T\left(\frac{2n}{3}\right) + 1$$

Step 1: Identify a , b , and $f(n)$. Compare $T(n) = 2T(2n/3) + 1$ with the standard form $T(n) = aT(n/b) + f(n)$. Note that $\frac{2n}{3} = \frac{n}{3/2}$, so the shrink factor is $b = 3/2$:

$$a = 2, \quad b = \frac{3}{2} = 1.5, \quad f(n) = 1.$$

Step 2: Compute the critical exponent $\log_b a$.

$$\log_b a = \log_{3/2} 2.$$

Numerically,

$$\log_{3/2} 2 = \frac{\ln 2}{\ln 1.5} = \frac{0.6931}{0.4055} \approx 1.710.$$

So the critical function is

$$n^{\log_b a} = n^{\log_{3/2} 2} \approx n^{1.710}.$$

Step 3: Compare $f(n)$ with $n^{\log_b a}$. Here $f(n) = 1 = n^0$. We compare the exponents:

$$\underbrace{0}_{\text{exponent of } f(n)} < \underbrace{1.710}_{\log_{3/2} 2}.$$

Since the exponent of $f(n)$ is *strictly smaller* than $\log_{3/2} 2$, $f(n)$ grows polynomially *slower* than the critical function. This points to **Case 1**.

Step 4: Verify the Case 1 condition formally. We must find an $\varepsilon > 0$ such that

$$f(n) = O\left(n^{\log_{3/2} 2 - \varepsilon}\right).$$

Take, for example, $\varepsilon = 0.5$. Then

$$n^{\log_{3/2} 2 - \varepsilon} = n^{1.710 - 0.5} = n^{1.210}.$$

Since

$$f(n) = n^0 = 1 \leq n^{1.210} \quad \text{for all } n \geq 1,$$

we have $f(n) = O(n^{1.210}) = O(n^{\log_{3/2} 2 - \varepsilon})$. The Case 1 condition holds. (Any ε with $0 < \varepsilon \leq \log_{3/2} 2 \approx 1.710$ works.)

Step 5: Apply Case 1 to get the answer. By Case 1 of the Master Theorem,

$$T(n) = \Theta\left(n^{\log_b a}\right) = \Theta\left(n^{\log_{3/2} 2}\right) \approx \Theta\left(n^{1.710}\right).$$

$$T(n) = \Theta\left(n^{\log_{3/2} 2}\right) \approx \Theta\left(n^{1.710}\right)$$